

Compiler Design

①

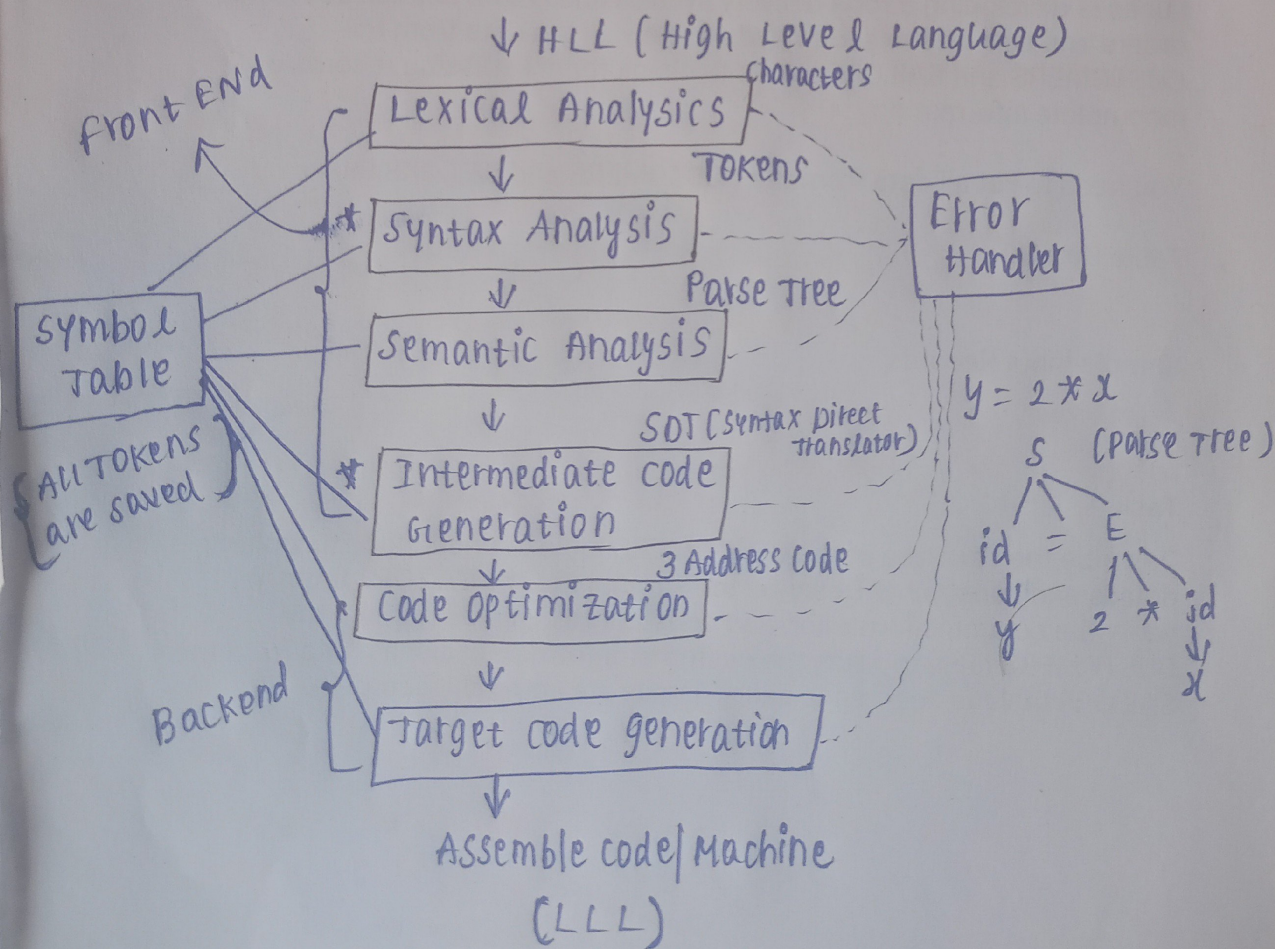
1. Lexical Analysis.
2. parser (syntax Analyzer).***
3. Semantic Analysis.
4. Intermediate code generator.*
5. code optimization.

②

Phases of compiler Design:

Compiler:

That is used to convert the High Level Language to the Low Level Language.



FRONT END

(2)

Lexical Analysis :

That converts the High Level Language to the Lexical Tokens

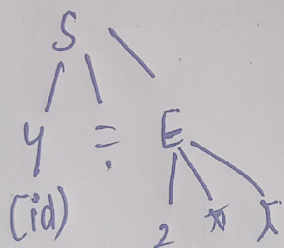
Syntax Analysis :

It analyze the ~~the~~ Token what is Identifier, & assignment operator [or] constants using the phase tree

Ex :

$y = 2 * x$, $\rightarrow E$ (Assume)

Phase Tree



$2, x, y \Rightarrow$ constant.

"=" $\Rightarrow$ assignment operator.

"*" $\Rightarrow$ Arithmetic operator.

Semantic Analysis :

What ever errors we are getting in the compile time

$\therefore$ using "SDT"

Intermediate code Generator :

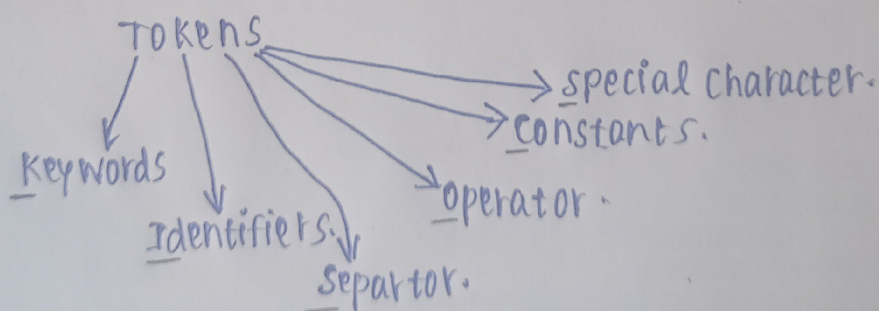
It generates an 3 Address code.

BackendCode Optimization :

same like an Time & space complexity.

Target code generation :That Generates the Target code That is
Machine code.NOTE :Frontend same for every Machine, Backend varies
depend on the Machine.Lexical Analysis :* Lexical Analysis are also called { lexer, tokenizer,
scanner }

1. Tokenization



2. Give ERROR Message

- Exceeding length.
- unmatched string.
- illegal special characters.

3. Eliminate comments, white space, blankes.

Ex :

(4)

```
int main()
```

```
{
```

```
/* find Max of a and b */
```

→ skip This part. becz it is an comment.

```
int a=1, b=4;
```

```
if(a>b)
```

```
return (a);
```

32 → Tokens

```
else
```

```
return (b);
```

```
}
```

NOTE :

Whether the Length how in the double quote we treated as token Length 1

Ex :

```
printf("i=%d j=%d k=%d", i, j, k);
```

11 → Tokens

Question on Lexical Analysis

Ex 1:

```
int main() {
```

```
{
```

```
x = y + z;
```

```
int x, y, z;
```

```
printf("sum of %d %d", x);
```

```
}
```

NOTE:
Only convert into the token, NOT checking the syntax

25 → Tokens

(5)

NOTE:

uninfix and Assignment Treated as one "Token"

EX:

main()

```

{
  a = b ++ ++ -- -- ++ ++ = j
  printf(" %d id %d", a, b);
}

```

25 → TOKENS

if having single
quote it will
stop

EX: 3:

```

main() {
  int a = 10;
  char b = "abc";
  int t c = 30;
  char ch ard = "xyz";
  in /* comment */ t m = 40.5;
}

```

32

32 Tokens

Finding first() & follow():

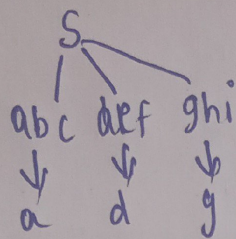
(small letters = terminal)

First(A):

Contains All terminals present in first place
of every ~~string~~ derived string.

- (1) $S \rightarrow abc(def)ghi$
- (2) $\text{first}(\text{terminal}) = \text{terminal}$
- (3) $\text{first}(\epsilon) = \epsilon$

Ex: $S \rightarrow abc|def|ghi$



For $First(S) = \{a, d, g\}$

Ex: 2:

$S \rightarrow ABC|ghi|JKL$

$A \rightarrow a|b|c$

$B \rightarrow b$

$D \rightarrow d$

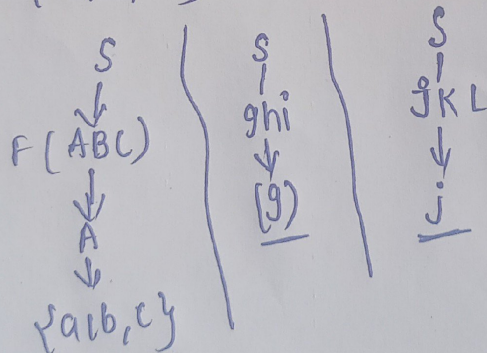
Sol:

$First(D) = \{d\}$

$first(B) = \{b\}$

$first(A) = \{a, b, c\}$

$first(S) =$



Ex: 3:

clear cut

$S \rightarrow ABC$

$A \rightarrow a|b|\epsilon$

$B \rightarrow c|d|\epsilon$

$C \rightarrow e|f|\epsilon$

Capital: variable
small: terminals

(6)

In CFG left hand side is always an variable.

$$\text{first}(C) = \{e, f, \epsilon\}$$

$$\text{first}(B) = \{c, d, \epsilon\}$$

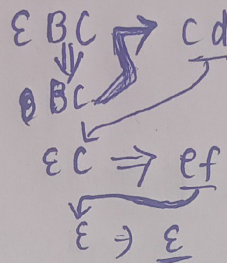
$$\text{first}(A) = \{a, b, \epsilon\}$$

$$\text{first}(S) = S$$

$$\downarrow$$

$$f(ABC)$$

$$f(A) = \{a, b, \epsilon\} \rightarrow \text{for } \epsilon \text{ we need to put in the main gramer.}$$



$$f(S) = \boxed{abcde}$$

EX 4:

$$E \rightarrow TE' \quad \boxed{id, C}$$

$$E' \rightarrow *TE' / \epsilon \quad \boxed{*, \epsilon}$$

$$T \rightarrow FT' \quad \boxed{id, C}$$

$$T' \rightarrow \epsilon / +FT' \quad \boxed{\epsilon, +}$$

$$F \rightarrow id / CE : \boxed{id, C}$$

$$\text{FIRST}(E) \rightarrow \boxed{id, C}$$

if " ϵ " means need to put & recursive

Brackets also terminal

Follow(A) : → check only Right side

(8)

Contains all terminals present immediate in right of "A"

Rules :

* Follow of start symbol(S) is \$

$$FO(A) = \$$$

2. $S \rightarrow ACD$

$$C \rightarrow a|b$$

$$FO(A) = \text{first}(C) = \{a, b\}$$

$$FO(D) = \text{Follow}(S) = \{\$, \}$$

3. $S \rightarrow asbs|bsas|\epsilon$

$$\text{follow}(S) = \{\$, b, a, \}$$

* follow Never contains the " ϵ "

* if right side having the Non-Terminal then = first(Non-Terminal)

* Last follow i.e Last letter the redirect to the Grammar

EX :

$$S \rightarrow ABC$$

$$A \rightarrow \epsilon$$

$$B \rightarrow \epsilon$$

$$C \rightarrow \epsilon$$

$$\text{Follow}(A) = \text{first}(CB) = \{\epsilon, \}$$

$$\Rightarrow A\epsilon C$$

$$\Rightarrow A\epsilon$$

$$\text{Follow}(A) = \text{first}(C) = \{\epsilon\}$$

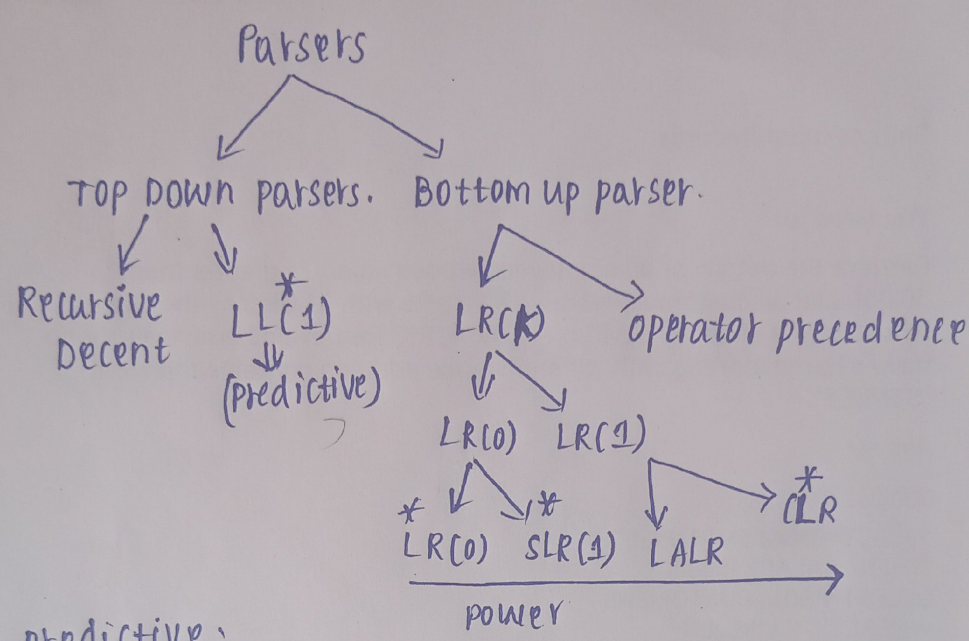
$$\Rightarrow A\epsilon$$

$$\Rightarrow A$$

$$S \rightarrow A$$

$$\text{Follow}(A) = \$$$

What is parsing & types of parsers / Syntax Analysis



LL(1) predictive:

- Goes With Left to Right.

parsing:

It is an process of deriving string from a given Grammar.

Ambiguous Grammar:

All the parser except the operator precedence

works in unambiguous Grammar

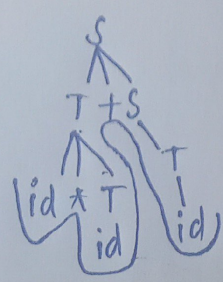
Ex:

$$S \rightarrow T + S / T$$

$$T \rightarrow id * T / id (s)$$

$$W = id * id + id$$

id * id + id



(10)

How to check whether the grammar is LL(1) or NOT?

$$S \rightarrow \overset{1}{(L)} \mid \overset{2}{a}$$

$$L \rightarrow SL' \quad 3$$

$$L' \rightarrow \underset{4}{\epsilon} \mid \underset{5}{,SL'}$$

First

$$F(S) = \{ (, a \}$$

$$F(L) = \{ (, a \}$$

$$F(L') = \{ \epsilon, , \}$$

↓ → terminal
follow

	(	)	a	,	\$
S	1		2		
L	3		3		
L'		4		5	

FOLLOW

$$F(S) = \{ \$,), , \}$$

$$F(L) = \{) \}$$

$$F(L') = \{) \}$$

\$ always

if " ϵ " at last then
follow of ϵ
 $L' \rightarrow \epsilon$
follow(L')

Always go with the $F(\text{first})$ in the grammar.

②

$$S \rightarrow \overset{1}{a}S\overset{2}{b}s \mid \overset{3}{b}S\overset{4}{a}s \mid \epsilon$$

$$\text{first}(S) = \{ a, b, \epsilon \}$$

$$\text{follow}(S) = \{ \$, a, b \}$$

	a	b	\$
S	1/3	2/3	3

$$S \rightarrow \epsilon$$

$$\text{follow}(S) = \{ \$, a, b \} = \text{③}$$

Checking of LL(1) Grammar Short Trick:

(11)

$$S \rightarrow aSa | bS | c$$

$$S \rightarrow \alpha_1 | \alpha_2 | \alpha_3$$

→ if no production is NOT 'NULL'

then

$$\text{find } \text{first}(\alpha_1) \cap \text{first}(\alpha_2) \cap \text{first}(\alpha_3) = \emptyset \quad \text{LL(1)}$$

$$\neq \emptyset \quad (\text{Not LL(1)})$$

→ If product is 'NULL'

$$S \rightarrow \alpha_1 | \alpha_2 | \epsilon$$

$$\text{first}(\alpha_1) \cap \text{first}(\alpha_2) \cap \text{follow}(S) = \emptyset \quad \text{LL(1)}$$

$$\neq \emptyset \quad (\text{Not LL(1)})$$

Ex:

$$S \rightarrow aSa | bS | c$$

$$\begin{array}{ccc} \downarrow & \downarrow & \downarrow \\ \text{first} = a & b & c \end{array}$$

$= \emptyset$ {No common elements}

$$a \cap b \cap c = \emptyset \Rightarrow \text{LL(1)} \checkmark$$

$$(2) S \rightarrow i(t^1 S_1^1)^2 | a$$

$$\Rightarrow i \cap a = \emptyset$$

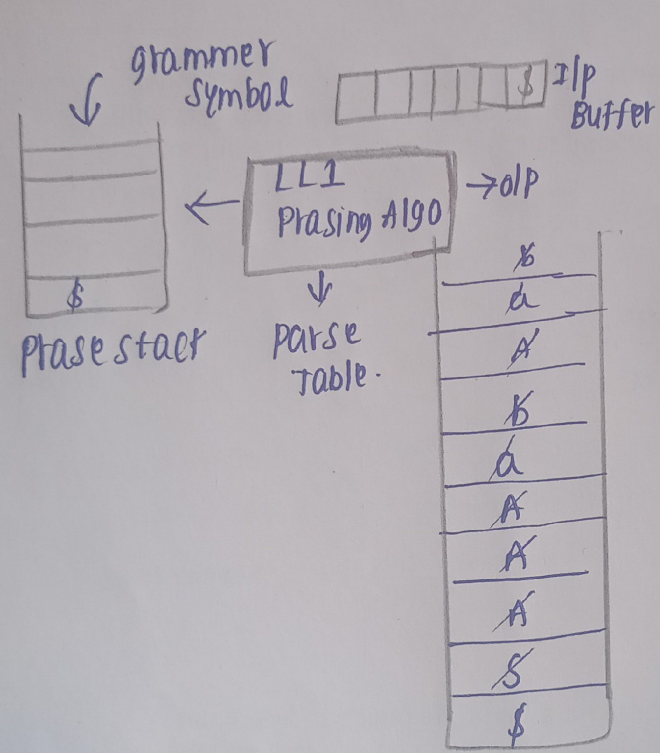
$$S_1 \rightarrow \epsilon^3 | \epsilon$$

$$\Rightarrow \epsilon \cap \epsilon \neq \emptyset \quad \text{Not LL(1)}$$

$$C \rightarrow b^5$$

What is LL(1) Parser? How it works?

- * It is an Top-down parser.
- * Left to Right Scan
- * Works on Left Most Derivation



$S \rightarrow AA$ 1
 $A \rightarrow aA$ 2
 $A \rightarrow b$ 3

Input Buffer: $a|b|a|b|b|$
 ↑
 Lookhead

	a	b	\$
S	1	3	
A	2	3	

Accepted by LR(0) parsing Table?

LR(K) = Canonical Items [C.I.]

Rule 1:

- * Make Duplicate and argument.
- * Write all production's with argument.

Bottom up approach

seen not seen

(dot) is last
 we say it is
 accept
 for duplicate

If we get actual
 right side grammar
 we called it reduce

Q:

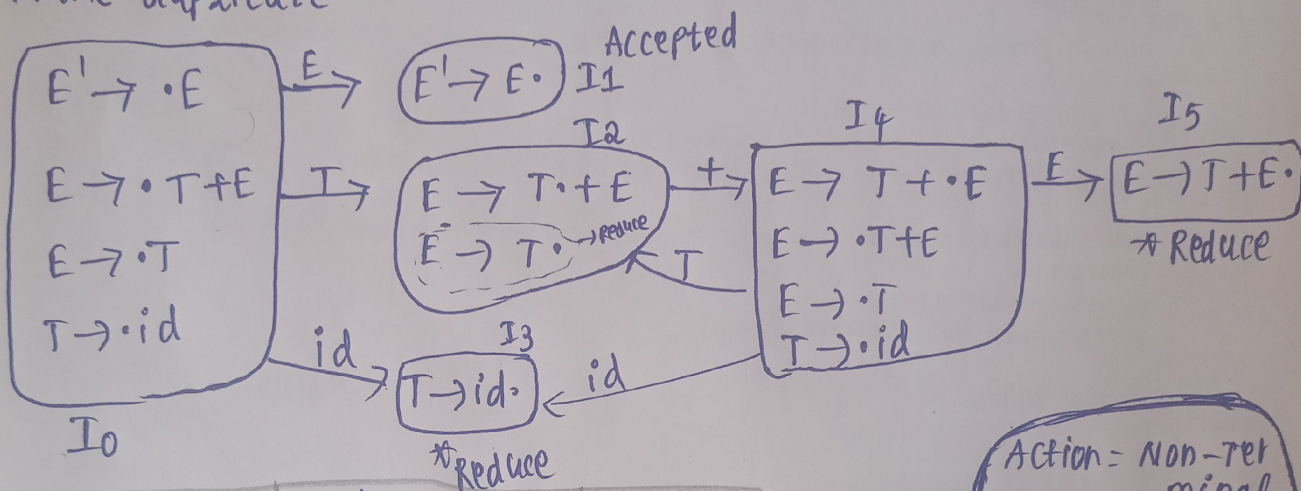
$E \rightarrow T^1 + E^2 / T$

$T \rightarrow id^3$

(13)

if $\cdot E$ getting
write all product-
ion of the $\cdot E$

Make duplicate



State	Action			GOTO	
	id	+	\$	E	T
0	S3		Accepted	1	2
1	-	-	Accepted	-	-
2	r2	S4 r2	r2		
3	S3 r3	S3 r3	S3 r3		
4	S3			5	2
5	r1	r1	r1		

Action = Non-terminal
GOTO = Terminal

Comparing with

$I_0 \xrightarrow{T} I_2$
 $I_0 \xrightarrow{E} I_4$

Bcz $T \cdot$

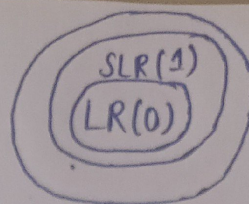
T is reduce to
 $E \rightarrow T \rightarrow 2$

For check:

Shift/Reduce (or) Reduce/Reduce

occurs any in on cell i.e SR (or) RR then
it is Not an LLR(1)

SLR(1)



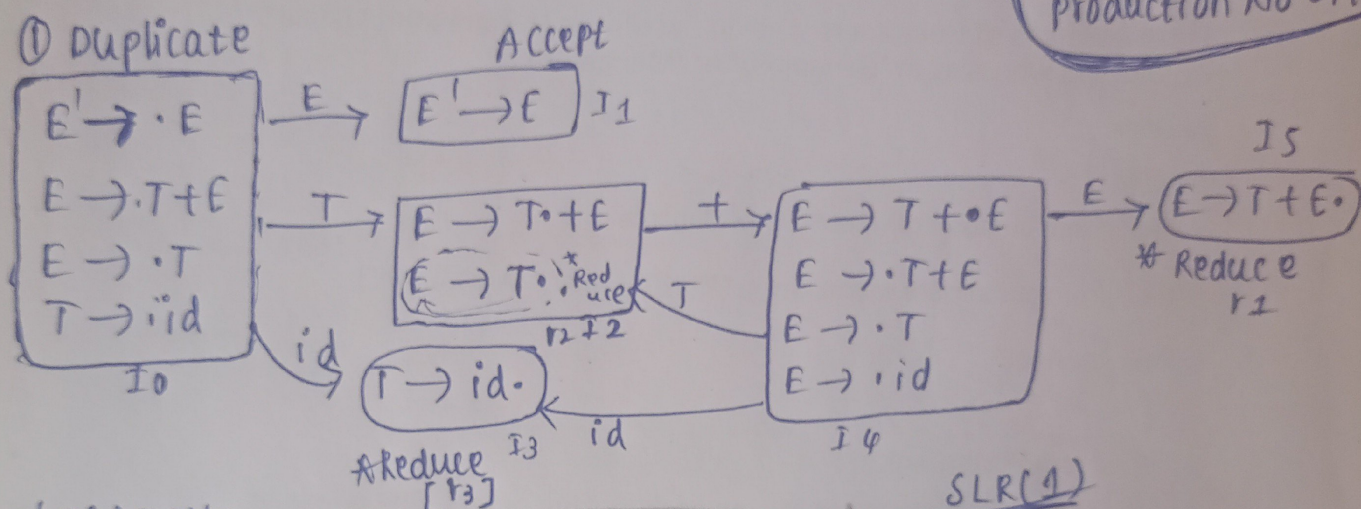
(14)

Q:

$$\begin{array}{l|l} E \rightarrow T^1 + E^2 & \text{follow} \\ T \rightarrow id^3 & \end{array} \quad \begin{array}{l} E = \{ \$ \} \\ T = \{ \$, + \} \end{array}$$

LR(0)

① Duplicate



for (LR(0) Table)
if Reduce write the suffix production No of it

LR(0) Table

State	Action			GOTO	
	\$	+	id	E	T
0			S3	1	2
1	Accepted				
2	r2	$\frac{S4}{r2}$	r3		
3	r3	r3	r3		
4			S3	5	2
5	r1	r1	r1		

SLR(1)

	id	+	\$
0	S3		
1	Accepted S4		
2		S4	r2
3		r3	r3
4	S3		
5			r1

for SLR(1) Table:

if have reduce the find the left side arrow follow of it.

CLR Parsing Table (LR 1 Concial items)

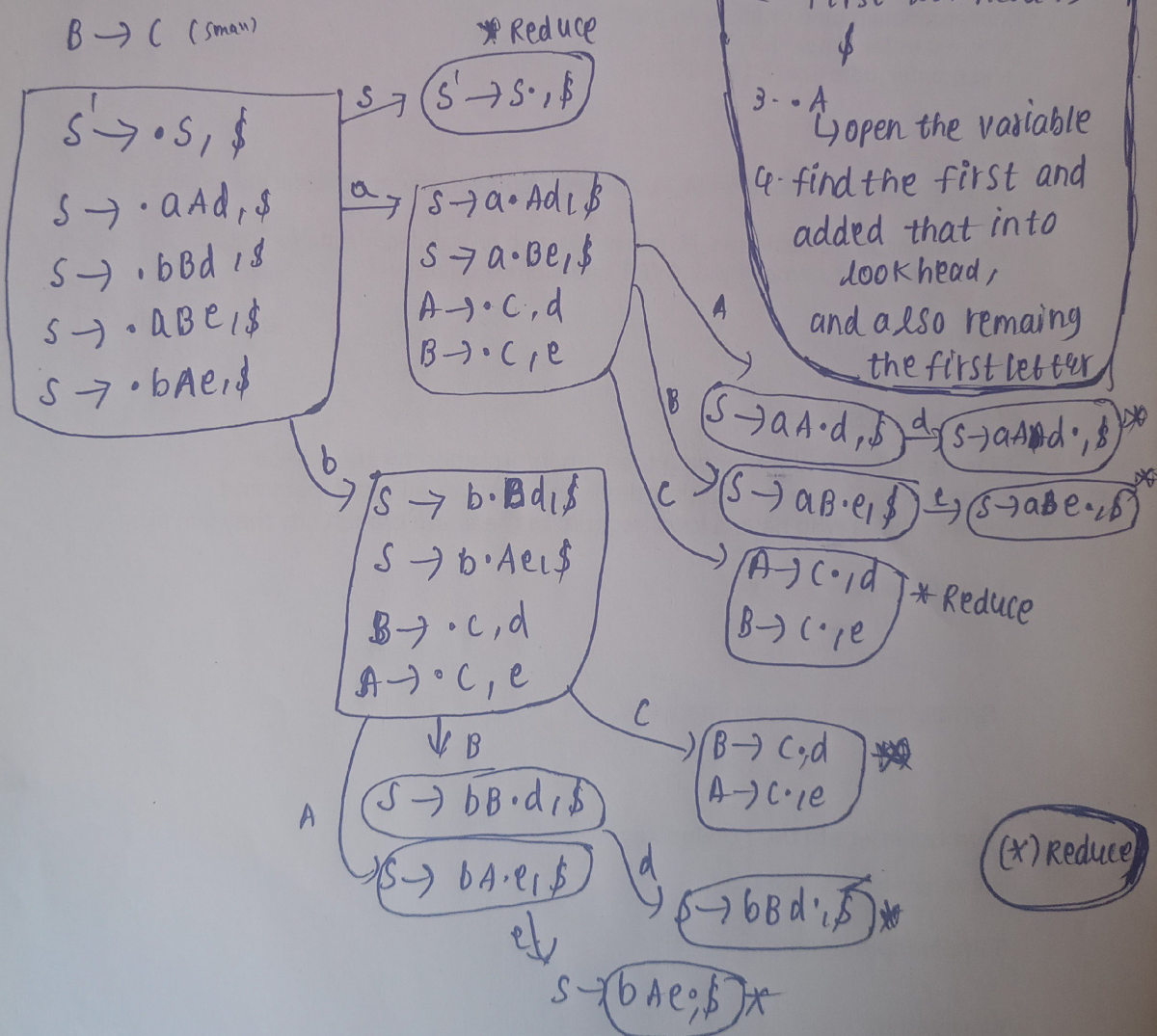
(15)

Q :

$S \rightarrow aAd / bBd / aBe / bAe$

$A \rightarrow C$ (small)

$B \rightarrow C$ (small)



CLR :

For Reduce we need to write at Look head symbol in the CLR Table

(No SR, RR) also table same as SLR

LALR VS CLRLALR < CLR

* Any question comes from the LALR. We need to find the CLR first.

LALR :

In "CLR" we find same grammar with different look head
We can merge both

Ex :

$A \rightarrow \underline{c} ; \underline{d}$

$A \rightarrow \underline{(} \cdot \underline{) e}$

die different